

Instituto Tecnológico Superior de Rio verde SLP.

Semana10

ALUMNO:

Brayan Ivanov Martinez Martinez

NUMERO DE CONTROL:

252240031

MATERIA:

Programación

DOCENTE:

Collazo

GRUPO

2^a A

Fecha

28/abril/2026

INDICE

1. Introducción
2. Planteamiento del problema
3. Diseño de la solución
4. Desarrollo código Evidencia de ejecución
5. Análisis del programa
6. Conclusiones

INTRODUCCIÓN

La programación orientada a objetos (POO) es una forma de programar que se usa mucho porque ayuda a que los programas estén más ordenados y sean más fáciles de entender. En lugar de hacer todo junto, se trabaja con clases y objetos que representan cosas de la vida real, y eso hace que el programa sea más claro y más fácil de modificar después.

Uno de los temas más importantes de la POO es la herencia. La herencia sirve para que una clase pueda usar cosas de otra clase que ya existe, como sus datos y funciones. Esto ayuda a no repetir código y a que todo tenga más sentido dentro del programa, ya que se pueden hacer relaciones entre clases.

En programación, la herencia es muy útil cuando hay cosas que se parecen pero no son exactamente iguales. Por ejemplo, en vez de hacer muchas clases desde cero, se puede hacer una general con lo básico y luego otras más específicas que agreguen o cambien algunas cosas. Así el código queda más limpio y es más fácil hacer cambios después.

En esta práctica se usa la herencia para hacer un sistema de empleados. Todos los empleados tienen cosas en común como nombre, edad y salario, pero también hay diferentes tipos. Algunos trabajan tiempo completo y tienen bonos, otros trabajan por horas, y también hay quienes tienen prestaciones como seguro o vacaciones.

El objetivo de esta práctica es entender cómo usar la herencia para organizar mejor el programa sin repetir código. También se busca practicar otras cosas como constructores, modificar métodos y organizar bien las clases.

Al final, este tipo de ejercicios ayuda a mejorar la lógica al programar y a entender cómo usar estos temas en la vida real. La herencia no solo sirve para la escuela, también se usa mucho cuando se hacen programas más grandes y completos.

PLANTEAMIENTO DEL PROBLEMA

El problema consiste en desarrollar un programa en C++ que permita representar diferentes tipos de empleados dentro de una empresa.

Todos los empleados tienen información básica como nombre, edad y salario. Sin embargo, no todos se comportan igual, ya que algunos trabajan tiempo completo y reciben un bono, mientras que otros trabajan por horas y su salario depende del tiempo trabajado.

Además, existen empleados que cuentan con prestaciones adicionales como seguro y vacaciones, lo que hace necesario agregar más características al modelo.

El objetivo principal es aplicar la herencia para organizar estas diferencias sin repetir código, creando una estructura clara que permita representar cada tipo de empleado de forma adecuada.

DISEÑO DE LA SOLUCIÓN

Antes de comenzar a programar, se analizó el problema para identificar qué elementos eran comunes y cuáles eran diferentes entre los empleados. Con base en esto, se decidió utilizar una clase base y varias clases derivadas.

Clase base: Empleado

Se creó una clase base llamada **Empleado**, la cual contiene los atributos comunes:

- Nombre
- Edad
- Salario

También incluye métodos como:

- Mostrar información
- Calcular salario

Clases derivadas

Se implementaron dos clases derivadas:

EmpleadoTiempoCompleto

- Hereda de Empleado
- Agrega el atributo bono
- Sobrescribe el método de salario para incluir el bono

EmpleadoPorHoras

- Hereda de Empleado
- Incluye horas trabajadas y pago por hora
- Calcula el salario en función de estos valores

Herencia múltiple

Se creó una clase adicional llamada **Prestaciones**, que contiene:

- Seguro
- Días de vacaciones

Posteriormente, se creó la clase:

EmpleadoCompletoConPrestaciones

- Hereda de EmpleadoTiempoCompleto y Prestaciones
- Combina salario con beneficios adicionales **Relación entre clases**
- Empleado → clase base
- EmpleadoTiempoCompleto → hereda de Empleado
- EmpleadoPorHoras → hereda de Empleado
- Prestaciones → clase independiente
- EmpleadoCompletoConPrestaciones → herencia múltiple

Este diseño permite reutilizar código y extender funcionalidades fácilmente.

DESARROLLO

El código fue desarrollado paso a paso, comenzando por la clase base y posteriormente agregando las clases derivadas. Durante el proceso se realizaron pruebas para verificar que el cálculo del salario funcionara correctamente en cada tipo de empleado.

Constructor

Cada clase tiene un constructor que inicializa sus atributos. En las clases derivadas, se utiliza la herencia para llamar al constructor de la clase base.

Herencia

Se utiliza la sintaxis:

```
class EmpleadoTiempoCompleto : public Empleado
```

Esto indica que la clase hereda los atributos y métodos de la clase base.

Override

Se utiliza la palabra clave `override` para redefinir métodos heredados, como el cálculo del salario, permitiendo que cada tipo de empleado tenga un comportamiento distinto.

Métodos

Cada clase tiene métodos para mostrar información y calcular el salario, adaptados según el tipo de empleado.

Función `main()`

En la función principal se crean objetos de cada tipo de empleado:

- Empleado base
- Empleado de tiempo completo
- Empleado por horas
- Empleado con prestaciones

Posteriormente, se muestran sus datos y se calcula su salario

ANÁLISIS DEL PROGRAMA

El uso de herencia facilitó mucho la organización del programa, ya que permitió definir una sola vez los atributos comunes y reutilizarlos en las demás clases. Esto hizo que el código fuera más limpio y entendible.

Si no se hubiera utilizado herencia, habría sido necesario repetir varias partes del código en cada tipo de empleado, lo cual aumentaría la probabilidad de errores.

Una de las dificultades fue entender cómo estructurar correctamente las clases y cuándo era necesario sobrescribir métodos. Sin embargo, una vez comprendido esto, el desarrollo del programa resultó más sencillo.

También se observó que la herencia múltiple puede ser útil, pero debe utilizarse con cuidado para no complicar demasiado el diseño.

CONCLUSIONES

En esta práctica entendí mejor cómo funciona la herencia en programación. Básicamente vi que puedes tener una clase principal y de ahí sacar otras más específicas sin tener que escribir todo otra vez, y eso hace que el código quede más ordenado.

También noté que ayuda bastante a no repetir cosas y ahorrar tiempo. Si después quieres cambiar algo o agregar más cosas, no se vuelve un caos porque todo ya tiene una estructura clara.

Otra cosa que aprendí fue cómo cambiar algunos métodos para que cada clase haga lo suyo, por ejemplo en cómo se calcula el sueldo dependiendo del tipo de empleado.

En general, esta práctica me sirvió para pensar mejor la lógica al programar y darme cuenta que es importante organizar bien las clases desde el inicio. Así el programa queda más fácil de entender y de modificar después.